

DotNet Technology

BCA — Semester 5 — 2020 — Answer Sheet

Subject Code: CACS302

Group B

Attempt any SIX questions. [6×5=30]

2. Differentiate Object Oriented Programming and Object Based Programming. Explain some of the major features of C# language.

[2+3]

Answer: Object Oriented Programming (OOP) vs Object Based Programming (OBP):

Aspect	Object Oriented Programming	Object Based Programming
Inheritance	Supports inheritance (single & multiple)	Does not support inheritance
Polymorphism	Supports polymorphism (overloading, overriding)	Limited or no polymorphism
Encapsulation	Full encapsulation support	Supports encapsulation
Examples	C#, Java, C++, Python, JavaScript, VBScript, Ruby (pre-2.0)	
Reusability	High reusability through inheritance	Limited reusability through composition only
Complexity	More complex, structured approach	Simpler, more flexible

Major Features of C# Language:

- 1. Object-Oriented:** Supports encapsulation, inheritance, and polymorphism. Everything is an object.
- 2. Type-Safe:** Strong typing prevents type errors at compile time. Variables must be declared with specific types.
- 3. Modern:** Supports generics, LINQ, async/await, lambda expressions, and extension methods.
- 4. Managed Code:** Runs on .NET CLR with automatic memory management (garbage collection).
- 5. Component-Oriented:** Supports properties, events, attributes, and reflection for building reusable components.
- 6. Interoperability:** Can interact with COM components, native C/C++ libraries (P/Invoke), and other .NET languages.
- 7. Versioning:** Supports method overloading and overriding with virtual/override keywords for safe versioning.
- 8. Standardized:** ECMA and ISO standardized language with cross-platform support via .NET Core/.NET 5+.
- 9. Rich Library:** Extensive Base Class Library (BCL) for common tasks like I/O, networking, collections, and threading.

3. Explain overview of Microsoft .NET framework and its components in detail.

[5]

Answer: Microsoft .NET Framework Overview: The .NET Framework is a software development platform developed by Microsoft. It provides a controlled programming environment where software can be developed, installed, and executed on Windows-based systems. It supports multiple programming languages and provides a large standard library. **Components of .NET Framework:** 1. **Common Language Runtime (CLR):** - The execution engine of .NET - Provides memory management (Garbage Collection) - Handles exception management, type safety, and security - Just-In-Time (JIT) compilation converts IL code to native machine code - Provides cross-language integration 2. **.NET Framework Class Library (FCL):** - A comprehensive collection of reusable classes, interfaces, and value types - Provides functionality for: file I/O, database connectivity, XML processing, web services, GUI development, cryptography, threading - Organized into namespaces (System, System.IO, System.Data, System.Web, etc.) 3. **Common Type System (CTS):** - Defines how types are declared, used, and managed - Ensures type safety and cross-language interoperability - Supports value types (structs, enums) and reference types (classes, interfaces, delegates) 4. **Common Language Specification (CLS):** - A subset of CTS that all .NET languages must support - Ensures interoperability between different .NET languages - Any CLS-compliant code can be used by any .NET language 5. **Base Class Library (BCL):** - Core subset of FCL providing fundamental functionality - Includes collections, I/O, threading, reflection, networking, and security 6. **ASP.NET:** - Web development framework for building web applications and services - Supports Web Forms, MVC, Web API, and SignalR - Provides server controls, data binding, and state management 7. **ADO.NET:** - Data access technology for connecting to databases - Supports disconnected data architecture with DataSets and DataAdapters - Provides providers for SQL Server, Oracle, OLE DB, and ODBC 8. **Windows Forms / WPF:** - Technologies for building desktop applications - Windows Forms: traditional event-driven GUI framework - WPF: modern UI framework using XAML for declarative UI design **Architecture:** Application Layer WinForms / WPF / ASP.NET / Console Base Class Library System, System.IO, System.Data, System.Web, etc. Common Language Runtime JIT Compiler, GC, Security, Type Checker Operating System Windows OS

4. What do you mean by property in C# language? How it is different from method? Compare automatic property with other types of property with suitable example.

[1+1+3]

Answer: Property in C#: A property in C# is a member that provides a flexible mechanism to read, write, or compute the value of a private field. Properties use accessors (get and set) to control access to fields. They appear like fields from the outside but are implemented like methods internally. **Property vs Method:** Aspect Property Method Syntax Accessed like a field: obj.Name Called with parentheses: obj.GetName() Purpose Encapsulates field access Performs operations/actions Return Always returns a single value Can return void or any type Parameters Cannot have parameters (except indexers) Can have multiple parameters Overloading Cannot be overloaded Can be overloaded Use Case Data encapsulation, validation Business logic, calculations **Types of Properties:** 1. **Read-Write Property (Full Property):** private string name; public string Name { get { return name; } set { name = value; } } 2. **Read-Only Property:** public string Email { get; } 3. **Write-Only Property:** public string Password { set { /* encrypt and store */ } } 4. **Automatic Property:** public string Name { get; set; } **Comparison: Automatic Property vs Full Property** Aspect Automatic Property Full Property Backing Field Compiler-generated, hidden Explicitly declared Validation Cannot add validation in basic form Can add validation logic in set Code Length Very concise More verbose Readability Clean and simple More explicit Initialization Can use default values Full control over initialization When to Use Simple data binding, DTOs Business logic, validation needed

5. Define constructor. Explain different types of constructors used in C# with example.

[1+4]

Answer: Constructor: A constructor is a special method in a class that is automatically invoked when an object of the class is created. It has the same name as the class and no return type. Constructors are used to initialize object state and allocate resources. **Types of Constructors in C#:** 1. **Default Constructor:** A constructor with no parameters. If no constructor is defined, C# provides a default constructor automatically. `public class Student { public string Name; public Student() // Default constructor { Name = "Unknown"; } }` 2. **Parameterized Constructor:** A constructor that accepts parameters to initialize object with specific values. `public class Student { public string Name; public int Age; public Student(string name, int age) { Name = name; Age = age; } }` // Usage: `Student s = new Student("Ram", 20);` 3. **Copy Constructor:** Creates a new object by copying values from an existing object. `public class Student { public string Name; public Student(Student other) { Name = other.Name; } }` 4. **Static Constructor:** Called automatically before any instance is created or static member is referenced. Used to initialize static data. `public class Database { public static string ConnectionString; static Database() { ConnectionString = "Server=localhost;Database=Test"; } }` 5. **Private Constructor:** Prevents instantiation from outside the class. Used in singleton pattern and utility classes. `public class Singleton { private static Singleton instance; private Singleton() { } public static Singleton GetInstance() { if (instance == null) instance = new Singleton(); return instance; } }` **Constructor Chaining:** Constructors can call other constructors using the `this` keyword: `public Student() : this("Unknown", 0) { } public Student(string name, int age) { Name = name; Age = age; }`

6. Define inheritance. Write a C# program to demonstrate multilevel and multiple inheritance.

[1+2+2]

Answer: Inheritance: Inheritance is an object-oriented programming concept where a new class (derived/child class) inherits properties and methods from an existing class (base/parent class). It promotes code reusability and establishes an "is-a" relationship between classes. **C# supports:** • Single inheritance (one base class per derived class) • Multilevel inheritance (class B inherits from A, class C inherits from B) • Hierarchical inheritance (multiple classes inherit from one base) • Multiple inheritance of interfaces (not classes) **Multilevel Inheritance Example:** // Base class `public class Person { public string Name; public void DisplayPerson() { Console.WriteLine("Name: " + Name); } }` // Intermediate derived class `public class Employee : Person { public int EmployeeId; public void DisplayEmployee() { DisplayPerson(); Console.WriteLine("Employee ID: " + EmployeeId); } }` // Derived from Employee `public class Manager : Employee { public string Department; public void DisplayManager() { DisplayEmployee(); Console.WriteLine("Department: " + Department); } }` // Usage class `Program { static void Main() { Manager mgr = new Manager(); mgr.Name = "Hari"; mgr.EmployeeId = 101; mgr.Department = "IT"; mgr.DisplayManager(); } }` **Multiple Inheritance using Interfaces:** C# does not support multiple class inheritance, but it supports multiple interface inheritance: `public interface IPrintable { void Print(); } public interface IScannable { void Scan(); } public class AllInOnePrinter : IPrintable, IScannable { public void Print() { Console.WriteLine("Printing document..."); } public void Scan() { Console.WriteLine("Scanning document..."); } }` class `Program { static void Main() { AllInOnePrinter device = new AllInOnePrinter(); device.Print(); device.Scan(); } }` **Output:** Name: Hari Employee ID: 101 Department: IT Printing document... Scanning document...

7. What is generics? List different types of generic classes. Explain delegate with example.

[1+1+3]

Answer: Generics: Generics in C# allow you to define type-safe data structures and methods without committing to actual data types at definition time. They enable code reusability by creating classes, interfaces, and methods that work with any data type while maintaining type safety at compile time. **Benefits of Generics:** • Type safety: Errors caught at compile time, not runtime • Performance: No boxing/unboxing for value types • Code reusability: One class/method works with multiple types • Reduced code duplication **Types of Generic Classes:** 1. **Generic Collections:** - List<T>: Dynamic array - Dictionary<TKey, TValue>: Key-value pairs - Queue<T>, Stack<T>: FIFO and LIFO structures 2. **Generic Interfaces:** - IEnumerable<T>, IList<T>, ICollection<T> 3. **Custom Generic Classes:** public class Box<T> { private T item; public void SetItem(T value) { item = value; } public T GetItem() { return item; } } // Usage: Box<int> intBox = new Box<int>(); 4. **Generic Methods:** public T Max<T>(T a, T b) where T : IComparable { return a.CompareTo(b) > 0 ? a : b; } 5. **Generic Constraints:** - where T : class (reference type) - where T : struct (value type) - where T : new() (parameterless constructor) - where T : BaseClass (inheritance constraint) - where T : IComparable (interface constraint) **Delegate:** A delegate is a type-safe function pointer that references methods. It defines the signature of methods that can be assigned to it. **Example:** // Delegate declaration public delegate void GreetingDelegate(string name); public class Program { public static void EnglishGreeting(string name) { Console.WriteLine("Hello, " + name); } public static void NepaliGreeting(string name) { Console.WriteLine("Namaste, " + name); } static void Main() { GreetingDelegate greet; greet = EnglishGreeting; greet("Ram"); // Output: Hello, Ram greet = NepaliGreeting; greet("Sita"); // Output: Namaste, Sita } } **Built-in Delegates:** • Action<T>: Delegate with no return value • Func<T, TResult>: Delegate with return value • Predicate<T>: Delegate that returns boolean

8. What do you mean by lambda expression? Explain different types of lambda expression used in C# with example.

[1+4]

Answer: Lambda Expression: A lambda expression is an anonymous function that can contain expressions and statements. It provides a concise way to write inline code blocks without explicitly declaring a method. Introduced in C# 3.0, lambdas are extensively used with LINQ, delegates, and event handlers. **Syntax:** (parameters) => expression or (parameters) => { statements; } **Types of Lambda Expressions:** 1. **Expression Lambda:** Contains a single expression on the right side of => operator. Returns the result of the expression. // Expression lambda with one parameter Func<int, int> square = x => x * x; Console.WriteLine(square(5)); // Output: 25 // Expression lambda with two parameters Func<int, int, int> add = (a, b) => a + b; Console.WriteLine(add(3, 4)); // Output: 7 2. **Statement Lambda:** Contains a block of statements enclosed in braces. Can include multiple statements, variable declarations, and loops. Action<string> greet = name => { string message = "Hello, " + name; Console.WriteLine(message); }; greet("Ram"); // Output: Hello, Ram Func<int, int, string> compare = (a, b) => { if (a > b) return a + " is greater"; else if (a < b) return b + " is greater"; else return "Both are equal"; }; Console.WriteLine(compare(5, 3)); // Output: 5 is greater 3. **Lambda with LINQ:** List<int> numbers = new List<int> { 1, 2, 3, 4, 5, 6 }; // Using lambda with Where var evenNumbers = numbers.Where(n => n % 2 == 0); foreach (var n in evenNumbers) Console.WriteLine(n + " "); // Output: 2 4 6 // Using lambda with Select var squares = numbers.Select(n => n * n); 4. **Lambda with Delegates:** delegate bool IsEligible(Student s); class Student { public string Name; public int Marks; } IsEligible isPass = s => s.Marks >= 40; Student student = new Student { Name = "Hari", Marks = 75 }; Console.WriteLine(isPass(student)); // Output: True 5. **Lambda with Events:** Button btn = new Button(); btn.Click += (sender, e) => { Console.WriteLine("Button clicked!"); }; **Key Points:** • Type inference: Compiler infers parameter types • Can capture variables from enclosing scope (closures) • Shorter and more readable than anonymous methods • Cannot be used where a statement block requires branching (return, break, continue) without statement lambda

Group C

Attempt any TWO questions. [2×10=20]

9. a) Write a program to create user registration form in one ASP.NET web page and display filled data in another page. b) Write a program for handling exception in ASP.NET.

[7+3]

Answer: a) ASP.NET User Registration Form: Registration.aspx (Form Page): <%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Registration.aspx.cs" Inherits="WebApp.Registration" %> <!DOCTYPE html> <html> <head><title>User Registration</title></head> <body> <form id="form1" runat="server"> <h2>User Registration</h2> <asp:Label ID="lblName" runat="server" Text="Name:"></asp:Label> <asp:TextBox ID="txtName" runat="server"></asp:TextBox>

 <asp:Label ID="lblEmail" runat="server" Text="Email:"></asp:Label> <asp:TextBox ID="txtEmail" runat="server"></asp:TextBox>

 <asp:Label ID="lblPhone" runat="server" Text="Phone:"></asp:Label> <asp:TextBox ID="txtPhone" runat="server"></asp:TextBox>

 <asp:Button ID="btnSubmit" runat="server" Text="Submit" OnClick="btnSubmit_Click" /> </form> </body> </html> **Registration.aspx.cs:** using System; namespace WebApp { public partial class Registration : System.Web.UI.Page { protected void btnSubmit_Click(object sender, EventArgs e) { Session["Name"] = txtName.Text; Session["Email"] = txtEmail.Text; Session["Phone"] = txtPhone.Text; Response.Redirect("Display.aspx"); } } } **Display.aspx (Result Page):** <%@ Page Language="C#" AutoEventWireup="true" CodeBehind="Display.aspx.cs" Inherits="WebApp.Display" %> <!DOCTYPE html> <html> <head><title>Display Data</title></head> <body> <form id="form1" runat="server"> <h2>Registered User Details</h2> <asp:Label ID="lblResult" runat="server"></asp:Label> </form> </body> </html> **Display.aspx.cs:** using System; namespace WebApp { public partial class Display : System.Web.UI.Page { protected void Page_Load(object sender, EventArgs e) { string name = Session["Name"].ToString(); string email = Session["Email"].ToString(); string phone = Session["Phone"].ToString(); lblResult.Text = "Name: " + name + "
Email: " + email + "
Phone: " + phone; } } } **b) Exception Handling in ASP.NET:** using System; namespace WebApp { public partial class ExceptionDemo : System.Web.UI.Page { protected void Page_Load(object sender, EventArgs e) { try { int a = 10; int b = 0; int result = a / b; // This will throw DivideByZeroException lblResult.Text = "Result: " + result; } catch (DivideByZeroException ex) { lblResult.Text = "Error: Cannot divide by zero. " + ex.Message; } catch (Exception ex) { lblResult.Text = "General Error: " + ex.Message; } finally { // Cleanup code always executes lblResult.Text += "
Execution completed."; } } } **Global Exception Handling (Global.asax):** void Application_Error(object sender, EventArgs e) { Exception ex = Server.GetLastError(); // Log error to file or database System.IO.File.AppendAllText("ErrorLog.txt", DateTime.Now + ": " + ex.Message + "\n"); Server.ClearError(); Response.Redirect("ErrorPage.aspx"); }

10. a) How virtual method is used to achieve dynamic binding in C#? Explain with the help of suitable program. b) Define operator overloading. Write a C# program to overload binary operator.

[1+4+1+4]

Answer: a) Virtual Method and Dynamic Binding: Dynamic Binding (Runtime Polymorphism): Dynamic binding is the process of resolving a method call at runtime rather than compile time. In C#, virtual methods enable dynamic binding by allowing a derived class to override a base class method. **How it works:** 1. Base class declares a method with virtual keyword 2. Derived class overrides it with override keyword 3. At runtime, the actual object's type (not the reference type) determines which method is called **Program:** using System; public class Shape { public virtual void Draw() { Console.WriteLine("Drawing a shape"); } } public class Circle : Shape { public override void Draw() { Console.WriteLine("Drawing a circle"); } } public class Rectangle : Shape { public override void Draw() { Console.WriteLine("Drawing a rectangle"); } } class Program { static void Main() { Shape s1 = new Circle(); // Upcasting Shape s2 = new Rectangle(); // Upcasting s1.Draw(); // Output: Drawing a circle s2.Draw(); // Output: Drawing a rectangle // Even though reference type is Shape, // the actual object's method is called at runtime } } **Key Points:** • Without virtual/override, method hiding occurs with new keyword • Dynamic binding enables extensibility and loose coupling • Used extensively in frameworks (ASP.NET MVC controllers, Entity Framework) **b) Operator Overloading:** Operator overloading allows custom types (classes/structs) to define how operators (+, -, *, /, ==, etc.) behave when applied to objects of that type. **Program to Overload Binary + Operator:** using System; public class Complex { public double Real { get; set; } public double Imaginary { get; set; } public Complex(double real, double imaginary) { Real = real; Imaginary = imaginary; } // Overload binary + operator public static Complex operator +(Complex c1, Complex c2) { return new Complex(c1.Real + c2.Real, c1.Imaginary + c2.Imaginary); } // Overload binary - operator public static Complex operator -(Complex c1, Complex c2) { return new Complex(c1.Real - c2.Real, c1.Imaginary - c2.Imaginary); } public override string ToString() { return Real + " + " + Imaginary + "i"; } } class Program { static void Main() { Complex c1 = new Complex(3, 4); Complex c2 = new Complex(1, 2); Complex sum = c1 + c2; Complex diff = c1 - c2; Console.WriteLine("c1 = " + c1); // Output: c1 = 3 + 4i Console.WriteLine("c2 = " + c2); // Output: c2 = 1 + 2i Console.WriteLine("Sum = " + sum); // Output: Sum = 4 + 6i Console.WriteLine("Diff = " + diff); // Output: Diff = 2 + 2i } } **Rules for Operator Overloading:** • Must be declared as public and static • At least one parameter must be of the containing type • Cannot overload =, &&, ||, ?:, new, is, as, sizeof, typeof operators • Can overload arithmetic (+, -, *, /, %), comparison (==, !=, <, >), and bitwise operators

11. a) What is LINQ? Write a program to select employees whose salary is greater than 20000 and whose address is kathmandu using LINQ. b) Write a C# program to show insert and select operation in database.

[1+4+5]

Answer: a) LINQ (Language Integrated Query): LINQ is a set of language extensions in C# that provides a consistent query syntax for querying data from various sources (objects, databases, XML, collections). It integrates query capabilities directly into the C# language, making queries type-safe and IntelliSense-supported. **LINQ Program:** using System; using System.Collections.Generic; using System.Linq; public class Employee { public int Id { get; set; } public string Name { get; set; } public string Address { get; set; } public decimal Salary { get; set; } } class Program { static void Main() { List<Employee> employees = new List<Employee> { new Employee { Id = 1, Name = "Ram", Address = "Kathmandu", Salary = 25000 }, new Employee { Id = 2, Name = "Sita", Address = "Pokhara", Salary = 30000 }, new Employee { Id = 3, Name = "Hari", Address = "Kathmandu", Salary = 18000 }, new Employee { Id = 4, Name = "Gita", Address = "Kathmandu", Salary = 35000 }, new Employee { Id = 5, Name = "Shyam", Address = "Lalitpur", Salary = 22000 } }; // LINQ Query Syntax var result = from emp in employees where emp.Salary > 20000 && emp.Address == "Kathmandu" select emp; // Alternative: Method Syntax // var result = employees.Where(emp => emp.Salary > 20000 && emp.Address == "Kathmandu"); Console.WriteLine("Employees with Salary > 20000 and Address = Kathmandu:"); foreach (var emp in result) { Console.WriteLine(\$"ID: {emp.Id}, Name: {emp.Name}, Salary: {emp.Salary}"); } } } **Output:** Employees with Salary > 20000 and Address = Kathmandu: ID: 1, Name: Ram, Salary: 25000 ID: 4, Name: Gita, Salary: 35000 **b) C# Database Operations (Insert and Select):** using System; using System.Data; using System.Data.SqlClient; public class DatabaseDemo { static string connString = "Server=localhost;Database=CompanyDB;Trusted_Connection=True;"; // INSERT Operation public static void InsertEmployee(string name, string address, decimal salary) { using (SqlConnection conn = new SqlConnection(connString)) { string query = "INSERT INTO Employees (Name, Address, Salary) VALUES (@Name, @Address, @Salary)"; SqlCommand cmd = new SqlCommand(query, conn); cmd.Parameters.AddWithValue("@Name", name); cmd.Parameters.AddWithValue("@Address", address); cmd.Parameters.AddWithValue("@Salary", salary); conn.Open(); int rowsAffected = cmd.ExecuteNonQuery(); Console.WriteLine(\$"{rowsAffected} row(s) inserted."); } } // SELECT Operation public static void SelectEmployees() { using (SqlConnection conn = new SqlConnection(connString)) { string query = "SELECT Id, Name, Address, Salary FROM Employees"; SqlCommand cmd = new SqlCommand(query, conn); conn.Open(); SqlDataReader reader = cmd.ExecuteReader(); Console.WriteLine("\nEmployee List:"); Console.WriteLine("ID\tName\tAddress\tSalary"); while (reader.Read()) { Console.WriteLine(\$"{reader["Id"]}\t{reader["Name"]}\t{reader["Address"]}\t{reader["Salary"]}"); } reader.Close(); } } static void Main() { // Insert a new employee InsertEmployee("Ram", "Kathmandu", 25000); // Display all employees SelectEmployees(); } } **Using DataAdapter and DataSet (Disconnected Architecture):** public static void SelectWithDataSet() { using (SqlConnection conn = new SqlConnection(connString)) { string query = "SELECT * FROM Employees"; SqlDataAdapter adapter = new SqlDataAdapter(query, conn); DataSet ds = new DataSet(); adapter.Fill(ds, "Employees"); foreach (DataRow row in ds.Tables["Employees"].Rows) { Console.WriteLine(\$"{row["Name"]} - {row["Salary"]}"); } } }

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.